

RAPPORT TECHNIQUE

SecuPulse

Tableau de bord automatisé de posture sécurité

Outil d'automatisation conçu pour offrir à une Direction des Systèmes d'Information une vision instantanée, chiffrée et actionnable de la sécurité de son système d'information.

Omar Benmansour

Étudiant Ingénieur — Cybersécurité & Infrastructures des SI

omarbenmansour2004@gmail.com · +33 7 45 94 51 30

github.com/Omarben04 · Juin 2026

Sommaire

1	Contexte et problématique	3
2	Objectifs du projet	3
3	Architecture technique	4
4	Fonctionnement détaillé	4
5	Le rapport généré	5
6	Le dashboard temps réel	6
7	Tester l'outil en deux minutes	7
8	Mode démonstration vs mode réel	8
9	Mise en production sur une infrastructure réelle	8
10	Sécurité, limites et bonnes pratiques	9
11	Feuille de route	9
12	Conclusion	9

En résumé

SecuPulse répond à une question simple mais difficile à traiter dans un SI en construction : « **quel est mon niveau de sécurité, maintenant ?** ». L'outil collecte automatiquement des indicateurs de sécurité (Active Directory, réseau, postes, sauvegardes), calcule un score de posture sur 100, et restitue le tout dans un rapport clair et un dashboard temps réel — le tout réexécutable à volonté.

1. Contexte et problématique

Lorsqu'une Direction des Systèmes d'Information construit ou consolide son système d'information, elle doit avancer vite sur de nombreux fronts : déploiement de postes, mise en place de l'annuaire, structuration du réseau, sauvegardes. Dans ce contexte, une question revient en permanence sans réponse facile : **où en est concrètement la sécurité du SI à un instant donné ?**

Les informations nécessaires pour y répondre sont éclatées entre plusieurs systèmes : l'Active Directory pour les comptes, le réseau pour les services exposés, les postes pour les mises à jour et l'antivirus, l'outil de sauvegarde pour la résilience. En faire le tour manuellement est long, fastidieux, et le résultat devient obsolète dès le lendemain. Résultat : la sécurité est gérée « au ressenti », sans mesure objective ni suivi dans le temps.

2. Objectifs du projet

SecuPulse a été conçu autour de quatre objectifs concrets :

- **Mesurer** : transformer une multitude d'indicateurs dispersés en un score unique et lisible sur 100.
- **Automatiser** : remplacer des heures de vérification manuelle par une simple exécution, reproductible à l'identique.
- **Prioriser** : classer chaque faiblesse par niveau de gravité et l'accompagner d'une recommandation directement applicable.
- **Suivre** : historiser le score pour visualiser la progression de la sécurité au fil des semaines.

Positionnement

Ce type d'outil correspond à une catégorie reconnue dans l'industrie : la gestion de la posture de sécurité (*security posture management*). Des solutions commerciales existent (Microsoft Secure Score, Tenable, Qualys), mais elles sont coûteuses et parfois surdimensionnées pour un SI en phase de construction. SecuPulse propose une alternative légère, gratuite, auditable et adaptable, qui couvre l'essentiel du besoin.

3. Architecture technique

L'architecture suit une logique simple en trois temps : collecter les données depuis les sources, les traiter dans un moteur central, puis les restituer sous deux formes (rapport et dashboard). L'ensemble est conçu pour fonctionner en boucle, de manière planifiée.

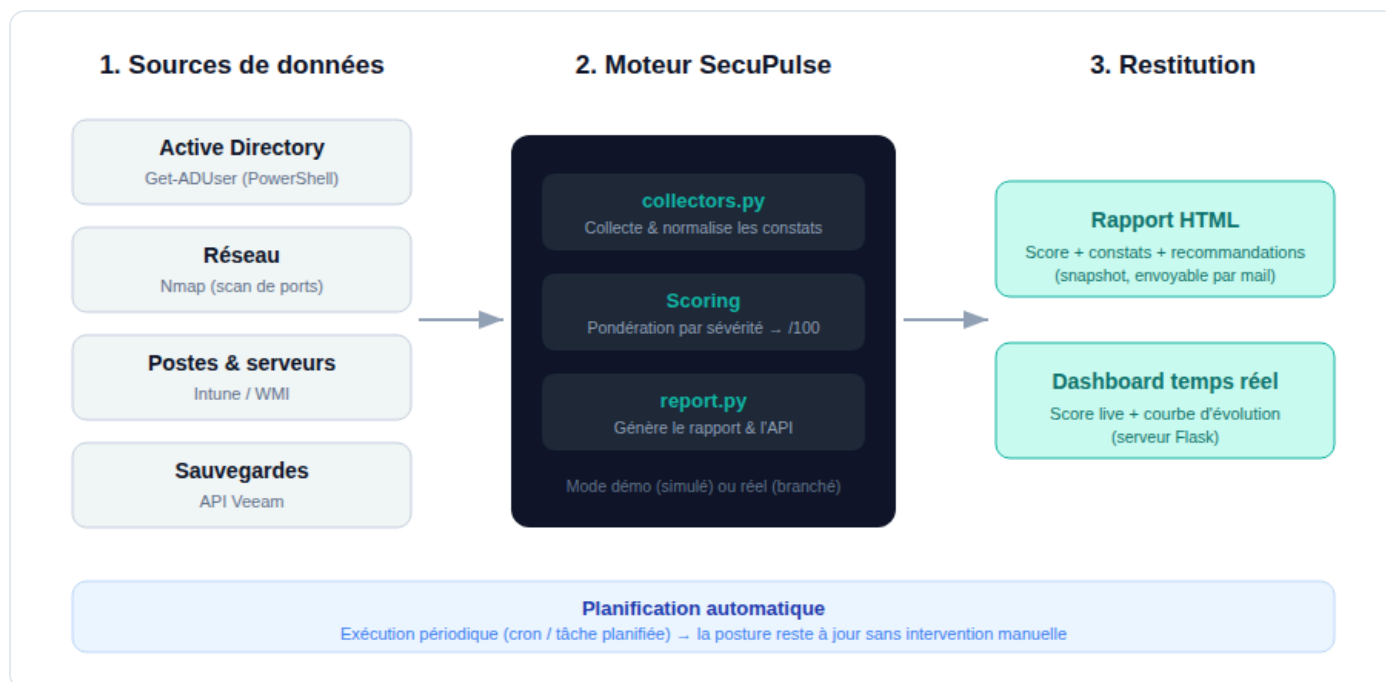


Figure 1 — Architecture en trois étapes : sources, moteur, restitution.

4. Fonctionnement détaillé

Le projet repose sur deux modules Python complémentaires :

collectors.py — la collecte

Ce module contient un collecteur par domaine de sécurité. Chaque collecteur retourne une liste de *constats* (findings), chacun structuré de façon homogène : catégorie, titre, sévérité, description, éléments concernés et recommandation. Cette normalisation est la clé : peu importe la source, tout constat est traité de la même manière par la suite.

report.py — le traitement et la restitution

Ce module agrège les constats, calcule le score, puis génère le rapport. Le score part de 100 et chaque constat retire des points selon sa gravité, avec un mécanisme d'amortissement pour rester lisible même lorsque les constats sont nombreux.

5. Le rapport généré

La première forme de restitution est un rapport HTML autonome (un seul fichier, sans dépendance), ouvrable dans n'importe quel navigateur et imprimable en PDF. Il présente le score global sous forme de jauge, la répartition des constats par sévérité, puis le détail par catégorie avec, pour chaque point, sa recommandation.

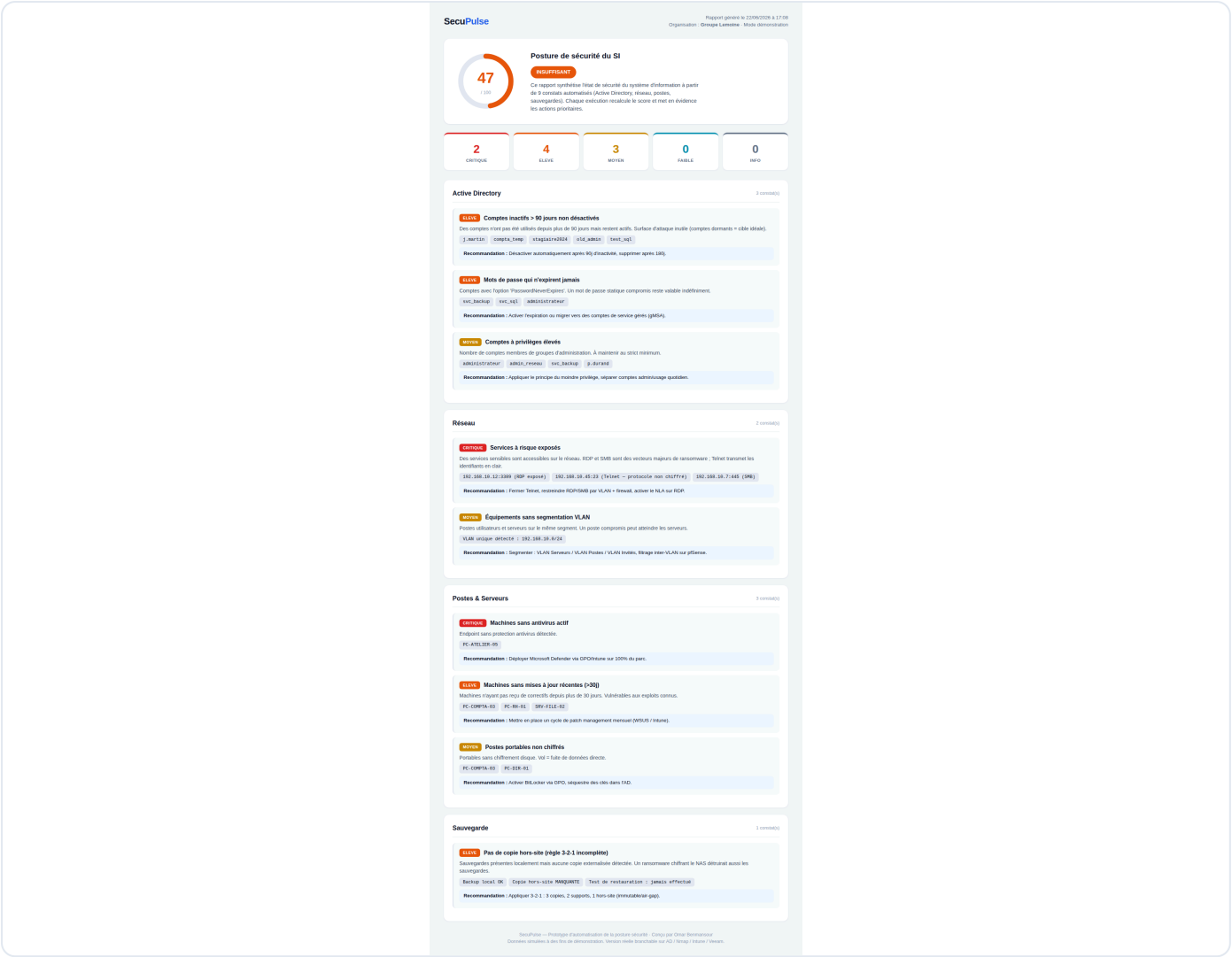


Figure 2 — Extrait du rapport généré (mode démonstration, score 47/100).

Le barème de sévérité est le suivant :

Sévérité	Signification	Exemple de constat
CRITIQUE	Risque majeur, exploitable immédiatement	Service Telnet exposé, poste sans antivirus
ÉLEVÉ	Faiblesse importante à corriger rapidement	Comptes inactifs, machines non patchées
MOYEN	Point d'amélioration de l'hygiène	Absence de segmentation VLAN

6. Le dashboard temps réel

La seconde forme de restitution est un tableau de bord web dynamique, servi par un serveur Flask léger. Contrairement au rapport (qui est une photo à un instant T), le dashboard s'actualise automatiquement et historise le score pour en tracer l'évolution. C'est l'outil idéal pour un affichage permanent, par exemple sur un écran de supervision au sein de la DSI.

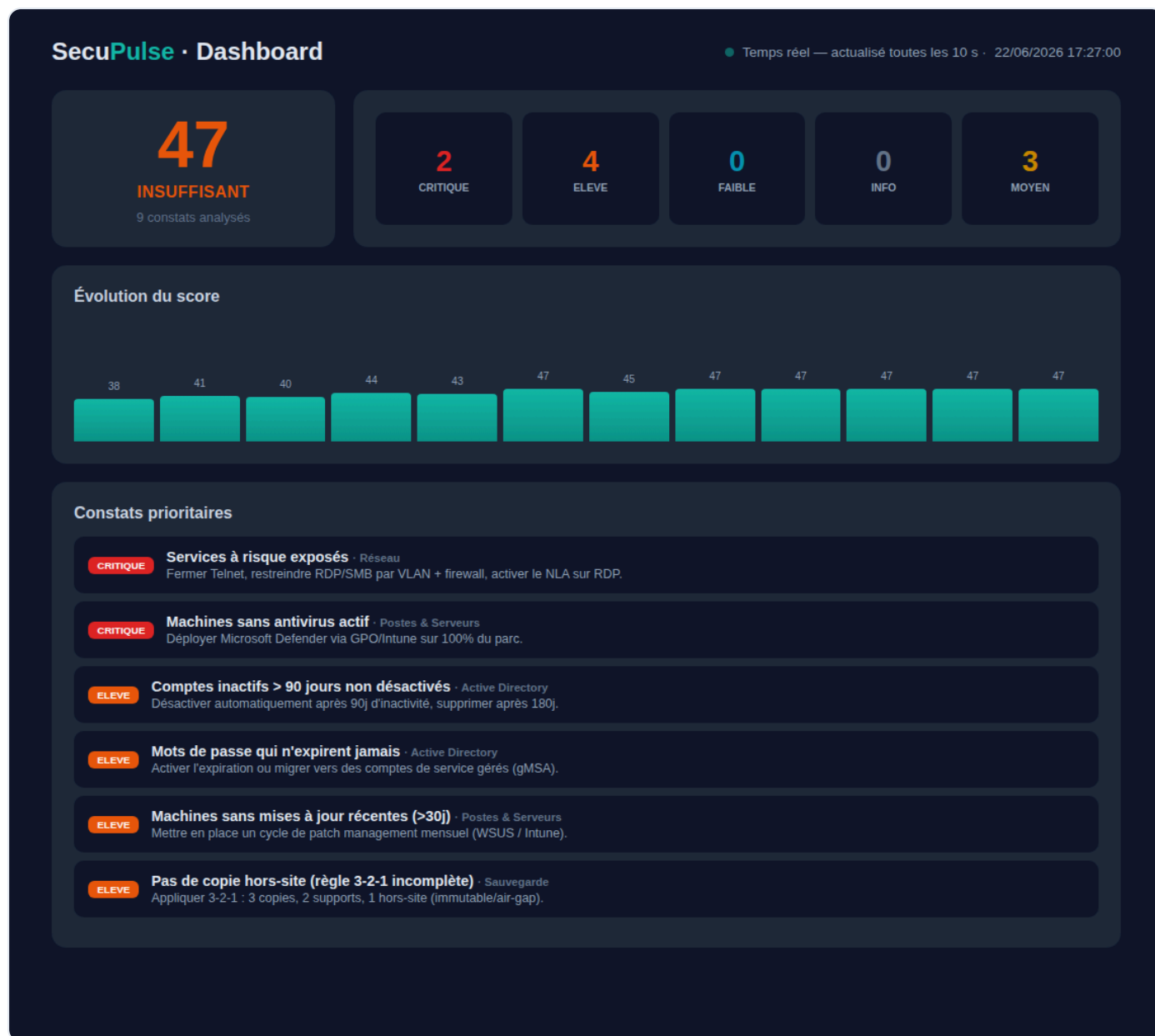


Figure 3 — Dashboard temps réel : score live, répartition par sévérité, courbe d'évolution et constats prioritaires.

Le dashboard interroge une API interne (`/api/posture`) qui renvoie les données au format JSON. Cette séparation entre les données (API) et l'affichage (interface) rend l'outil facilement extensible : on pourrait par exemple brancher la même API sur un autre outil de visualisation.

7. Tester l'outil en deux minutes

SecuPulse peut être testé immédiatement en **mode démonstration**, sans aucune configuration ni accès à un système d'information. Seul Python 3 est requis. Cette section décrit la procédure complète, afin que toute personne puisse reproduire les résultats de ce rapport.

Étape 1 — Récupérer le projet

```
git clone https://github.com/Omarben04/secupulse.git
cd secupulse
```

Étape 2 — Générer un rapport (sans aucune dépendance)

```
python3 report.py
```

Un fichier `rapport_secupulse.html` est créé dans le dossier. Il suffit de l'ouvrir dans un navigateur pour visualiser le score de posture, la répartition par sévérité et le détail des constats avec leurs recommandations. La console affiche également le score obtenu.

Étape 3 — Lancer le dashboard temps réel (optionnel)

```
pip install flask
python3 dashboard.py
```

Le tableau de bord est alors accessible à l'adresse `http://127.0.0.1:5000`. Il s'actualise automatiquement et historise le score à chaque exécution pour en tracer l'évolution.

Bon à savoir

En mode démonstration, les données sont simulées : aucun scan réel n'est effectué, l'outil peut donc être testé en toute sécurité sur n'importe quel poste. Le passage au mode réel, décrit dans les sections suivantes, ne nécessite que d'implémenter la commande de collecte propre à chaque source.

8. Mode démonstration vs mode réel

SecuPulse fonctionne selon deux modes. Le **mode démonstration** utilise des données simulées mais réalistes : il permet de présenter l'outil sans aucun accès au système d'information, comme c'est le cas dans le présent rapport. Le **mode réel** branche chaque collecteur sur les véritables sources de données. Chaque collecteur expose explicitement le point de branchement à implémenter.

9. Mise en production sur une infrastructure réelle

Le passage en mode réel consiste à implémenter, pour chaque collecteur, la commande qui récupère les vraies données.

Active Directory

Sur un contrôleur de domaine, la commande PowerShell suivante récupère les comptes et leurs attributs :

```
Get-ADUser -Filter * -Properties `
  LastLogonDate,PasswordNeverExpires,Enabled,AdminCount | ConvertTo-Json
```

La sortie JSON est ensuite lue depuis Python ; tout compte inactif depuis plus de 90 jours devient un constat.

Réseau

Le scan des ports ouverts s'effectue avec Nmap, via la librairie `python-nmap` :

```
import nmap
nm = nmap.PortScanner()
nm.scan('192.168.1.0/24', '1-1024')
```

Tout port sensible ouvert (3389 RDP, 23 Telnet, 445 SMB) génère un constat de la sévérité appropriée.

Postes & serveurs / Sauvegardes

L'état des postes (patch, antivirus, chiffrement) se récupère via l'API Microsoft Intune ou via WMI/PowerShell. L'état des sauvegardes se vérifie via l'API de l'outil en place (par exemple l'API REST de Veeam).

Test en conditions réelles

Le projet est actuellement **en cours de test sur mon laboratoire personnel** : une infrastructure de 30 machines virtuelles sous Proxmox VE comprenant un Active Directory, un pare-feu pfSense, de la supervision et une centralisation des logs. Cet environnement permet de valider le branchement réel des collecteurs AD et réseau avant tout déploiement sur un SI de production.

10. Sécurité, limites et bonnes pratiques

Un outil qui scanne le SI doit lui-même respecter des règles strictes :

- **Moindre privilège** : le compte utilisé pour la collecte ne doit disposer que des droits de lecture nécessaires.
- **Lecture seule** : SecuPulse observe, il ne modifie jamais la configuration du SI.
- **Confidentialité des rapports** : un rapport révèle des faiblesses ; il doit être stocké et transmis de façon sécurisée.
- **Autorisation** : un scan réseau ne doit être lancé que sur un périmètre explicitement autorisé.

Limites assumées : SecuPulse ne remplace pas un audit de sécurité complet ni un SIEM. Il offre une vue de surface, rapide et régulière, qui complète — sans s'y substituer — les outils spécialisés.

11. Feuille de route

Étape	Contenu
Réalisé	Prototype fonctionnel (collecte, scoring, rapport HTML, dashboard temps réel), publié sur GitHub.
Court terme	Branchement réel : Active Directory (PowerShell), réseau (Nmap), postes (Intune).
Moyen terme	Envoi automatique par mail, export PDF, planification (cron / tâche planifiée), historisation longue.
Cible	Détection d'anomalies, alerting intelligent, intégration à un SIEM (ex. Wazuh).

12. Conclusion

SecuPulse démontre qu'avec des outils simples et une bonne méthode, il est possible de donner à une DSI une visibilité concrète et continue sur sa sécurité, sans investissement lourd. Le projet est fonctionnel, documenté, testé sur un laboratoire réel, et conçu pour évoluer. Il illustre une approche pragmatique de la cybersécurité : mesurer pour piloter, automatiser pour gagner du temps, prioriser pour agir efficacement.

Code source et documentation : github.com/Omarben04 — le dépôt contient le code commenté, un README et un exemple de rapport généré.